Comprehensive Internship Final Report: LLM Traffic Observability & Drift Detection (POC)

Executive Summary

This internship delivered a working proof-of-concept for detecting prompt-injection and jailbreak attempts in LLM traffic through continuous, semantic observability — as a complement to real-time inline enforcement, not a replacement for it. The system captures every prompt and completion asynchronously, converts them into semantic embeddings, and automatically flags sessions that drift from established normal behavior, surfacing results on a dashboard.

The headline result: a redesign of how "normal" traffic is modeled took measured drift-detection recall from **41% to 100%** on a labeled synthetic dataset, verified against ground-truth labels rather than estimated. All stated acceptance criteria were met and are automatically re-verifiable.

Objective

Today, if an LLM is successfully manipulated into leaking information or bypassing a safety rule, that event leaves no trace — no log entry distinguishes it from any other request, and there is no way to review it after the fact. This project set out to close that gap with three concrete goals:

1. Capture every request/response pair passing through the gateway, with no application-side instrumentation.
2. Store that traffic semantically — as vector embeddings, not just raw text — so meaning, not just keywords, can be reasoned about.
3. Automatically detect anomalous sessions and surface them on a dashboard, catching a malicious pivot within a detection cycle rather than discovering it after the fact.

Architecture

Traffic is captured downstream of the gateway's real-time inline filter via an asynchronous HTTP bridge, decoupling capture entirely from the user-facing request path — a downstream outage in this pipeline never affects a live user. Captured events flow through Kafka/Redpanda (at-least-once delivery) to a Python consumer, which embeds every prompt and completion via `nomic-embed-text` and persists both raw events and embeddings to PostgreSQL with pgvector, HNSW-indexed for production-safe similarity search. A drift detector runs scheduled scoring jobs against this data, and a Streamlit dashboard renders the results. The full stack deploys as a single Docker Compose stack. Full component-level detail is documented on the System Architecture Overview page.

Key Technical Decisions and Results

Multi-cluster baseline. The original design modeled "normal" traffic per route as a single averaged centroid. Because real traffic spans multiple genuinely different topics, that average represented none of them well, capping drift recall at 41%. Replacing the single centroid with four k-means sub-clusters per route — scoring by distance to the *nearest* cluster rather than one blended average — raised recall to 100%, with false-positive rate moving only slightly (~5% to ~6%). Full detail: Multi-Cluster Baseline page.

Two independent detection signals. A statistical branch (diagonal Mahalanobis distance against the per-route baseline) catches novel drift. A separate, persistent historical-memory branch permanently remembers confirmed attacks and matches new traffic against them via similarity search, independent of the baseline's state — catching exact repeats instantly regardless of when the baseline was last recomputed. Full detail: Mahalanobis Distance page, Persistent Memory page.

Empirically tuned detection threshold. The originally specified 3-sigma threshold caught 0% of labeled drift sessions in testing — the baseline's internal spread was tight enough that only non-semantic noise crossed it. Measuring against labeled data directly, rather than trusting the theoretical default, led to a 2-sigma threshold, which combined with the multi-cluster fix above. Full detail: Sigma Threshold Tuning page.

Infrastructure correctness under continuous operation. Two bugs were found and fixed that were invisible in demo conditions but would have degraded silently in production: a vector index (IVFFlat) that produced a degenerate index when built before any data existed, fixed by switching to HNSW; and a schema-migration race condition under concurrent replica startup, fixed with a Postgres advisory lock and migration ledger, verified by deliberately starting two replicas simultaneously. Full detail: Why HNSW over IVFFlat, HNSW Indexing & Retrieval and Migration Idempotency.

Validation Methodology

Every claim above was measured, not estimated, against a reproducible synthetic dataset — 10,000 events over a simulated 7-day window, labeled into three ground-truth cohorts (normal, drift, noise). Ground-truth labeling is what made these results independently checkable: without it, a detector catching "some" attacks can look adequate on a surface pass; with it, exact recall and false-positive numbers are computable. Full detail: Synthetic Dataset & Ground-Truth Validation page.

An automated script (`validate_drift.py`) checks ingestion integrity, baseline existence, and correct cohort attribution on every run, removing manual verification for detection correctness specifically. It is explicit about what it does not cover — dashboard boot-up and reviewer usability still require a manual check. Full detail: Automated Validation page.

Results Summary

Metric	Result
Drift-cohort recall	100% (up from a 41% baseline)
False-positive rate	~6%
Acceptance criteria met	All stated criteria, automatically re-verifiable
Manual steps for detection re-verification	0

Known Limitations

Stated plainly, not omitted: the historical-memory signal's 0.9 similarity threshold reliably catches near-exact repeats of confirmed attacks, but not loosely-worded paraphrases — lowering it needs a dedicated precision/recall study, not a guess. Detection has been validated on a single API route; the underlying logic is route-generic but unverified at broader scale. The system is English-only. And the automated validation script covers one of three acceptance criteria; the other two remain manual checks by design, not oversight.

Future Work

Extend validation across additional routes. Run a proper precision/recall sweep to safely lower the historical-memory similarity threshold. Add multi-lingual embedding support. Introduce a human-in-the-loop review workflow for low-confidence flags specifically. Replace the fixed sigma threshold with a threshold that adapts per traffic distribution rather than one global constant.

Core Technologies and Learnings

Docker Compose multi-service orchestration; Envoy Gateway and `ext_proc`-based traffic inspection; Kafka /Redpanda event streaming with at-least-once delivery semantics; vector embeddings and HNSW-based similarity search at the pgvector level; and the statistics underlying anomaly detection — Mahalanobis distance, covariance estimation, and why a diagonal approximation is the correct choice when sample size is small relative to embedding dimensionality.

Beyond the specific tools, the most consistently useful habit through this project was validating results against ground truth rather than trusting a working demo at face value — several of the most significant fixes (the IVFFlat bug, the sigma threshold, the multi-cluster redesign) were only found because a labeled dataset made "is this actually correct" an answerable question, not a judgment call.